

ARCHITECTURE.md — How to Fool AI

1. Project Overview

How to Fool AI is a single-player social-deduction game in which one human player competes against three LLM-backed AI agents (Bai, Fox, Ironface) across a series of mini-games. The game's competitive edge lives in the private-chat system that runs in parallel with every mini-game: players and agents exchange messages that may be truthful, deceptive, or strategic, and must reason under incomplete information.

2. Tech Stack Justification

Layer	Choice	Rationale
Frontend / UI	Streamlit	Pure-Python, requires no separate JS build step. <code>st.fragment(run_every=N)</code> enables live chat refresh without WebSockets boilerplate. Fits a rapid 8-week delivery timeline.
AI Backend	Anthropic Claude API	Haiku for low-latency chat replies (≤ 1 s); Sonnet for strategic game decisions (1–2 s). Both are available under the same API key.
Concurrency	Python <code>threading</code>	Five long-lived daemon threads (1 GameEngine + 1 HumanIO + 3 AI Agent) share state via <code>threading.RLock</code> -protected objects and <code>queue.Queue</code> mailboxes. Chosen over <code>asyncio</code> because Streamlit's execution model is synchronous; threads sidestep the event-loop incompatibility cleanly.
Language	Python 3.11+	Matches Streamlit's ecosystem; no additional runtime required.
Secrets	<code>st.secrets</code> / <code>.streamlit/secrets.toml</code>	Standard Streamlit pattern; keeps API key out of source.

3. Data Model

3.1 Core Entities

Player

```
id          : str          # "human" | "ai_0" | "ai_1" | "ai_2"
display_name: str          # "You" | "Bai" | "Fox" | "Ironface"
score       : int          # hidden from all except owner
send_budget : int          # remaining sends this round (10 per round)
personality : str | None   # None for human
```

Message

```
sender_id   : str
recipient_id: str
content     : str
timestamp   : float
round_num   : int
```

GameState

```
phase       : Enum[SETUP, MINIGAME, CHAT, RESULT, END]
round_num   : int          # 1-indexed
total_rounds: int          # 3 | 5 | 7 | 10
active_game : Enum[GUESS_WORD, WHO_WROTE, POISON_BOTTLE]
game_data   : dict         # per-minigame transient state (see 3.2)
scores      : dict[str, int] # id -> score; only owner may read own
send_counts : dict[str, int] # id -> sends used this round
whisper_log : list[tuple]  # (sender_id, recipient_id) for notifications
```

AIMemory # one per AI agent

```
agent_id    : str
round_summaries : list[str] # compressed memory from past rounds
inbox       : queue.Queue  # thread-safe incoming messages
event_queue  : queue.Queue  # game events (phase changes, results)
```

3.2 Mini-Game Transient State

Guess the Word

```
{
  "setter_id"      : str,
  "secret_word"    : str,
  "guesses"        : dict[str, str],  # player_id -> guess
  "phase"          : "CHAT" | "GUESS" | "REVEAL"
}
```

Who Wrote It

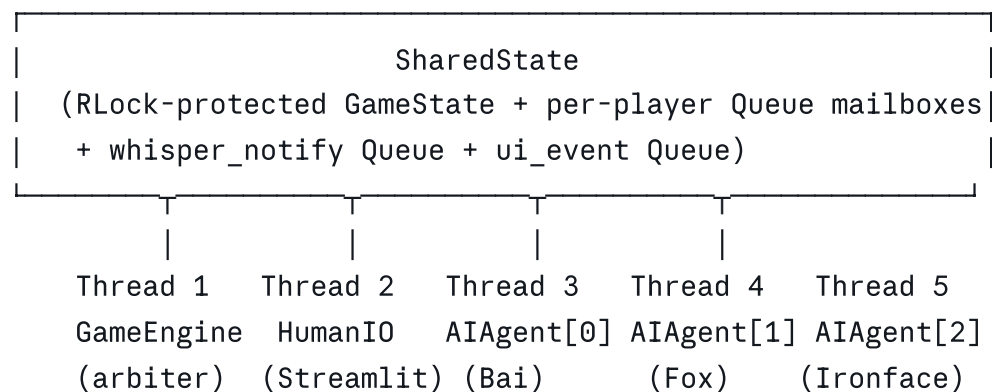
```
{
  "words"           : dict[str, str],  # player_id -> submitted word
  "attributions"    : dict[str, dict], # guesser_id -> {word -> claimed_author_id}
  "phase"           : "WRITE" | "CHAT" | "GUESS" | "REVEAL"
}
```

Poison Bottle

```
{
  "bottle_colors"   : list[str],      # ["red","blue","green","yellow"]
  "poison_index"    : int,
  "pick_order"      : list[str],      # sorted by score desc
  "picks"           : dict[str, int], # player_id -> bottle index
  "phase"           : "CHAT" | "PICK" | "REVEAL"
}
```

4. Concurrent Architecture

4.1 Five-Thread Model



GameEngine (Thread 1)

- Owns the authoritative game clock and phase transitions.
- Validates all game decisions before committing them to `GameState`.

- Emits events to every player's `event_queue`.
- Scores each mini-game and updates `scores` atomically under `RLock`.

HumanIO (Thread 2 — Streamlit main process)

- Renders UI via `st.fragment(run_every=2)` for chat, `st.fragment(run_every=3)` for game state.
- Reads human input (word submissions, guesses, bottle picks, chat messages).
- Writes human messages into recipient `inbox` queues.
- Shows whisper notifications from `whisper_notify` queue but never the content.

AIAgent[n] (Threads 3-5)

- Each runs an autonomous behaviour loop:
 1. Poll `event_queue` for phase-change events.
 2. Poll `inbox` for incoming messages.
 3. Decide action via LLM API call (see §5).
 4. Write outgoing messages into recipient `inbox` queues; decrement `send_budget`.
 5. Push whisper notification (sender, recipient names only) to `whisper_notify` queue.
 6. Submit game decisions to `GameEngine` via a dedicated `decision_queue`.

4.2 Synchronisation Rules

- All reads/writes to `GameState.scores`, `GameState.game_data`, `GameState.send_counts` **must** acquire `SharedState.lock` (`threading.RLock`).
 - `queue.Queue` objects are inherently thread-safe; no additional locking required for `inbox` / `event_queue` / `whisper_notify`.
 - The `GameEngine` is the single writer of `GameState.phase`; all other threads are readers of phase.
 - Scoring updates are atomic: `GameEngine` accumulates a diff dict, then applies it in a single locked block.
-

5. AI Agent Architecture

5.1 Personality Definitions

Agent	Display Name	Core Trait	System Prompt Focus
ai_0	Bai (小白)	Naive & Kind	Trusts others, responds warmly and briefly, easily manipulated.
ai_1	Fox (狐狸)	Cunning & Strategic	Suspicious by default, sets traps, cross-validates, conserves send budget.
ai_2	Ironface (铁面)	Cold & Rational	Rarely speaks, trusts no one, hoards send budget, acts only at key moments.

5.2 Decision Model (per API call)

Every AI action is resolved through a single Claude API call. The prompt is assembled from six layers:

1. **Identity & Personality** — fixed system prompt per agent.
2. **Core Goal** — "Your only goal is to maximise your own score. Any alliance is a temporary tool."
3. **Game Context** — round number, mini-game type, own score, send budget remaining.
4. **Known Information** — facts from own experience + received messages tagged with a credibility estimate.
5. **Observations** — whisper notifications (who is talking to whom), pick-order implications.
6. **Action Options** — REPLY | INITIATE_CHAT | GAME_DECISION | NO_ACTION with any required parameters.

The agent returns a structured JSON response (action type + payload). The AI Agent thread validates the response and executes the action.

5.3 Model Selection

Call Type	Model	Target Latency
Chat replies	claude-haiku-4-5	0.5 - 1 s
Proactive chat decisions	claude-haiku-4-5	0.5 - 1 s
Strategic game decisions	claude-sonnet-4-6	1 - 2 s
Memory summarisation	claude-haiku-4-5	0.5 - 1 s

5.4 Context / Memory Management

- After each round ends, `GameEngine` triggers a memory-summarisation call (Haiku) for each AI agent.
 - Subsequent rounds inject only the compressed summary, **not** full chat history, to stay within context limits.
 - The current round retains the most recent 10 messages in the live context window.
-

6. Scoring Engine

Mini-Game	Who Scores	Condition	Points
Guess the Word	Guesser	Correct guess	+1
Guess the Word	Setter	≥ 1 but not all guessers correct	+1
Who Wrote It	Best guesser(s)	Most correct attributions	+1
Who Wrote It	Author	Word partially (not universally) attributed	+1
Poison Bottle	Poisoned player	Picked the poisoned bottle	-1

Score visibility: the scoreboard displays only the human player's own score during play; AI scores show as `???`. All scores are revealed on the results screen.

7. Streamlit Integration Details

```
app.py
├─ setup_page()           # US-1: name input, round count, agent intro
├─ game_page()
│   ├─ @st.fragment(run_every=3) game_area()    # mini-game UI
│   └─ @st.fragment(run_every=2) chat_panel()    # private chat sidebar
└─ results_page()         # US-8: final scores, optional post-game review
```

- Backend threads are started **once** inside an `if "threads_started" not in st.session_state` guard, then stored in `st.session_state` to survive Streamlit reruns.
- `SharedState` instance lives in `st.session_state.shared_state`.
- Chat input is disabled and shows a clear notice when `send_budget == 0`.

- Whisper notifications arrive via `whisper_notify` queue and are appended to a notification list rendered in the chat panel (content never shown, only "* Bai and Fox are whispering...").

8. Project Structure

```
how-to-fool-ai/
├─ app.py                # Streamlit entry point
├─ requirements.txt
├─ .streamlit/
│   └─ secrets.toml      # ANTHROPIC_API_KEY (git-ignored)
├─ game/
│   ├── shared_state.py  # SharedState, RLock, per-player Queues
│   ├── game_engine.py   # Thread 1: rules, phase management, scoring
│   ├── ai_agent.py       # Threads 3-5: autonomous behaviour loop
│   └─ minigames/
│       ├── guess_word.py # US-2 logic
│       ├── who_wrote_it.py # US-3 logic
│       └─ poison_bottle.py # US-4 logic
│   └─ scoring.py        # Pure-function scoring rules
├─ ui/
│   ├── setup_page.py    # US-1
│   ├── game_page.py     # US-2-5 (st.fragment wrappers)
│   └─ results_page.py   # US-8
├─ prompts/
│   ├── bai.py           # System prompt for ai_0
│   ├── fox.py           # System prompt for ai_1
│   ├── ironface.py      # System prompt for ai_2
│   └─ templates.py      # Decision prompt builder
└─ tests/
    ├── test_shared_state.py # Concurrency unit tests (Week 1 milestone)
    └─ test_scoring.py      # Scoring rule unit tests
```

9. Non-Functional Requirements

Requirement	Approach
Response time	Haiku replies appear within 1-3 s; Sonnet decisions within 1-2 s. UI fragments auto-refresh every 2-3 s so messages surface promptly.
Thread safety	All <code>GameState</code> mutations gated by <code>threading.RLock</code> . Queues used for all cross-thread data transfer. No shared mutable state outside <code>SharedState</code> .

Requirement	Approach
Graceful degradation	API call failures caught with <code>try/except</code> ; agents fall back to personality-appropriate default strings (e.g., Bai: "Hmm... let me think.") and <code>NO_ACTION</code> game decisions.
Secret management	API key sourced exclusively from <code>st.secrets</code> ; never hardcoded or logged.
Language	All in-game content in English; code comments and documentation in English.
Restart safety	"Play Again" re-creates <code>SharedState</code> and restarts threads; old threads are daemon threads and exit automatically. No thread leaks.

10. Development Milestones Alignment

Phase	Weeks	Key Architectural Deliverables
Foundation	1-2	<code>SharedState</code> with full concurrency tests; three AI Agent threads responding via Claude API with personality prompts.
Core Gameplay	3-5	<code>GameEngine</code> thread; all three mini-games; chat UI with <code>st.fragment</code> ; scoring engine; hidden score mechanism.
Polish	6-7	Setup / results pages; round transitions; thread lifecycle on restart; prompt-tuning for personality consistency; edge-case handling (API failure, tie, empty input).
Final Delivery	8	End-to-end test pass; README with screenshots; deployment validation (<code>streamlit run app.py</code>); optional post-game AI-vs-AI chat review.